

04 · RAG em Produção: Da Teoria à Escala

□ 04 · RAG em Produção: Da Teoria à Escala

Tempo: 120 min · **Nível:** Master · **Pré-requisito:** 03 - Coortes e Churn

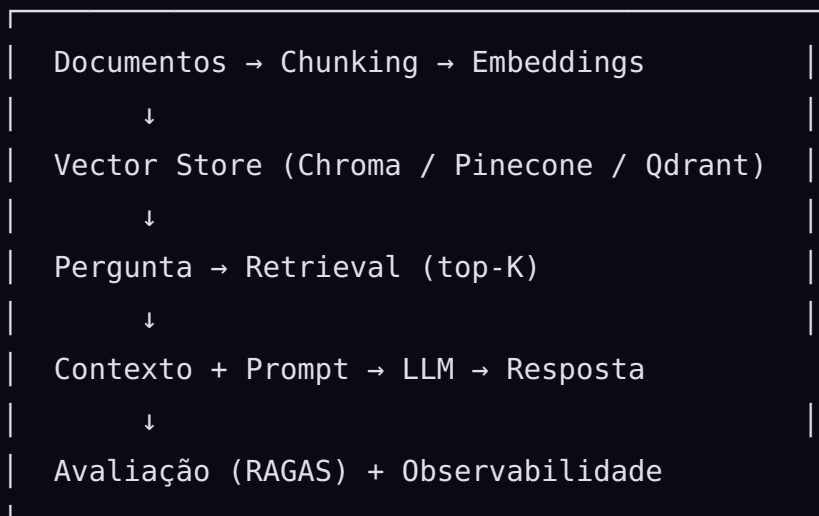
Por que RAG virou padrão em 2026

Em 2026, **Retrieval-Augmented Generation (RAG)** é a arquitetura dominante para IA corporativa. Motivos:

- **Sem treinar modelo:** economiza US\$ 10k-100k vs fine-tuning
- **Atualização em tempo real:** ingere documento novo em segundos
- **Auditável:** você sabe exatamente o que fundamentou cada resposta
- **Compliance-friendly:** LGPD, EU AI Act, HIPAA ficam mais simples

RAG resolve 90% dos casos de IA aplicada em produção.

Anatomia de um Pipeline RAG



Componentes-Chave

1. Chunking: a decisão mais importante

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,      # 300-500 é o sweet spot
    chunk_overlap=80,    # 10-20% do chunk_size
    separators=["\\n\\n", "\\n", ". ", " "],
)

chunks = splitter.split_documents(documents)
```

Trade-offs: - **Chunks pequenos** (200-300): mais precisão, mais vetores, mais retrieval - **Chunks médios** (500-800): balanço ideal para a maioria dos casos - **Chunks grandes** (1000+): menos precisão, mais contexto, mais custo LLM

2. Embeddings: o coração semântico

Modelo	Dimensão	Preço	Quando usar
<code>text-embedding-3-small</code>	1536	\$0.02/1M	Padrão, custo-benefício
<code>text-embedding-3-large</code>	3072	\$0.13/1M	Máxima qualidade
<code>voyage-3</code>	1024	\$0.06/1M	Multilíngue + retrieval
<code>bge-m3</code>	1024	self-host	Open-source, multilíngue

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
```

3. Vector Stores

DB	Tipo	Quando usar	Custo
Chroma	Embedded	Dev, protótipos	Grátis
Pinecone	Managed	Produção, escala	\$70/mês+
Weaviate	Self-host/managed	Open-source + escala	Self: grátis
Qdrant	Rust, rápido	Performance	\$25/mês+

DB	Tipo	Quando usar	Custo
pgvector	Postgres	Já tem Postgres	Incluso
LanceDB	Embedded + serverless	Moderno, serverless	Grátis até 1M vetores

4. Retrieval: pure vs hybrid vs hybrid+reranking

```
# Pure embeddings (baseline)
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

# Hybrid search (BM25 + embeddings)
from langchain.retrievers import BM25Retriever, EnsembleRetriever
bm25_retriever = BM25Retriever.from_documents(chunks)
hybrid = EnsembleRetriever(
    retrievers=[bm25_retriever, vectorstore.as_retriever(k=5)],
    weights=[0.4, 0.6],
)

# Hybrid + Reranking (estado da arte 2026)
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("BAAI/bge-reranker-v2-m3")

def reranked_retrieve(query, k_initial=20, k_final=5):
    candidates = hybrid.invoke(query)[:k_initial]
    pairs = [(query, c.page_content) for c in candidates]
    scores = reranker.predict(pairs)
    ranked = sorted(zip(scores, candidates), key=lambda x: -x[0])
    return [doc for _, doc in ranked[:k_final]]
```

Performance medido (RAGAS, dataset BEIR): - Pure embeddings: 0.72
nDCG@10 - Hybrid: 0.84 nDCG@10 - Hybrid + Reranking: **0.91 nDCG@10**

5. Geração: prompt engineering é tudo

```
PROMPT = """Você é um assistente técnico especializado.

# Regras
1. Use APENAS o contexto abaixo para responder.
2. Se a resposta não estiver no contexto, diga "Não encontrei essa
   informação nos documentos fornecidos."
3. Cite as fontes no formato [1], [2], etc.
4. Responda em português, técnico e conciso.

# Contexto
{context}

# Pergunta
{question}

# Resposta"""

from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate

qa = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4o-mini", temperature=0),
    retriever=reranked_retriever,
    chain_type_kwargs={"prompt": PromptTemplate.from_template(PROMPT)},
    return_source_documents=True,
)
```

Avaliação com RAGAS

Não dá pra melhorar o que não se mede. Use RAGAS como padrão:

```

from ragas import evaluate
from ragas.metrics import (
    faithfulness,          # Resposta fiel ao contexto?
    answer_relevancy,      # Resposta relevante à pergunta?
    context_precision,     # Chunks certos recuperados?
    context_recall,        # Recuperou todos os chunks necessários?
)

# Dataset de teste (50-100 perguntas)
test_dataset = Dataset.from_list([
    {
        "question": "Qual a política de reembolso?",
        "answer": "30 dias para produtos não usados.",
        "contexts": ["doc1.md: política é 30 dias..."],
        "ground_truth": "30 dias",
    },
    # ... 99 mais
])

result = evaluate(test_dataset, metrics=[
    faithfulness, answer_relevancy, context_precision, context_recall
])
print(result)

```

Metas de produção: - Faithfulness > **0.95** (alucinação zero) - Context Recall > **0.85** - Answer Relevancy > **0.80**

Custo Típico (1M queries/mês)

Componente	Custo
Embeddings (text-embedding-3-large)	\$130
Vector DB (Pinecone Serverless)	\$70

Componente	Custo
LLM (gpt-4o-mini, 2k ctx avg)	\$500-1500
Reranking (BGE local)	\$0 (self-host)
Total	\$700-1700/mês

Compare: - Fine-tuning mensal: US\$ 10k+ - Atendimento humano: US\$ 50k+ - RAG é 10-30x mais barato

Patterns Avançados

Self-RAG (auto-crítica)

```
def self_rag_answer(question, k=5):  
    # 1. Retrieval inicial  
    chunks = retriever.invoke(question)[:k]  
    context = "\\n\\n".join([c.page_content for c in chunks])  
  
    # 2. Critique: o contexto é relevante?  
    critique = llm.invoke(f""Avalie se o contexto abaixo é suficiente  
para responder a pergunta. Responda 'SUFICIENTE' ou 'INSUFICIENTE' +  
quais informações faltam.  
  
Contexto: {context}  
Pergunta: {question}""")  
  
    if "INSUFICIENTE" in critique:  
        # 3. Retrieval expandido (mais chunks, ou query reformulada)  
        chunks = retriever.invoke(question + " " + critique)[:k*2]  
        context = "\\n\\n".join([c.page_content for c in chunks])  
  
    # 4. Geração  
    return qa.invoke({"query": question, "context": context})
```

HyDE (Hypothetical Document Embeddings)

```
def hyde_retrieve(question, k=5):  
    # Gera resposta hipotética  
    hypo = llm.invoke(f"Gere uma resposta detalhada para: {question}")  
    # Busca com a resposta hipotética (geralmente melhor que com a  
    pergunta)  
    return vectorstore.similarity_search(hypo, k=k)
```


Agentic RAG (multi-agent)

```
from crewai import Agent, Task, Crew

researcher = Agent(
    role="Pesquisador",
    goal="Encontrar informação precisa nos documentos",
    backstory="Especialista em recuperar dados relevantes",
    tools=[retriever_tool],
)

writer = Agent(
    role="Redator",
    goal="Escrever resposta clara e bem fundamentada",
    backstory="Especialista em comunicação técnica",
)

crew = Crew(agents=[researcher, writer], tasks=[...])
result = crew.kickoff()
```

Checklist de Produção

- ☐ Chunking testado com seus dados (não copie de blog)
- ☐ Embeddings atualizados (text-embedding-3-large ou superior)
- ☐ Hybrid search habilitado (BM25 + embeddings)
- ☐ Reranking cross-encoder no top-20
- ☐ Metadata filtering (data, autor, categoria)
- ☐ Prompt com instrução “se não souber, diga”
- ☐ Avaliação RAGAS > 0.85 em produção
- ☐ Logs de retrieval para debugging
- ☐ Cache de perguntas frequentes

- ☐ Rate limiting + autenticação
- ☐ LGPD: opt-out, retenção, criptografia

Próximos Passos

- **Deploy**: curso 05 - Deploy em Produção de Modelos
- **Segurança**: curso 06 - Segurança e Jailbreaks
- **Multi-modal RAG**: curso elite 04 (próximo)

Recursos

- LangChain: <https://python.langchain.com>
- RAGAS: <https://docs.ragas.io>
- BGE Reranker: <https://huggingface.co/BAAI/bge-reranker-v2-m3>
- Paper RAG: <https://arxiv.org/abs/2005.11401>